Windows Security Log Analysis Report

Week 4 Lab: Windows Log Analysis with PowerShell

Field	Details
Analyst	EYASIN
Date	02 November 2026
System	ASUS, Windows 11, Windows PowerShell 5.1 (build 26100.9818)
Analysis Period	Last 1 hour / 24 hours / 7 days (as specified per exercise)
Scenario	SOC analyst at FinanceCorp investigating suspicious activity on Windows servers

Executive Summary

This lab used PowerShell (`Get-WinEvent`) to query the Windows Security Event Log for failed logons (4625), successful logons (4624), explicit credential use (4648), network share access (5140), group membership changes (4732) and special privilege assignments (4672).

Every filtered query returned **"No events were found that match the specified selection criteria"**, and the only unfiltered query (`Get-WinEvent -LogName Security -MaxEvents 10`) returned an **UnauthorizedAccessException**. As a result, no brute-force, lateral-movement or privilege-escalation activity could be confirmed or ruled out. The evidence points to a **permissions (non-elevated session) or audit-policy problem** rather than a clean system, and the queries should be re-run from an elevated PowerShell (see Troubleshooting section).

1. Environment and Prerequisites

Command:

```
$PSVersionTable.PSVersion
```

Result: Windows PowerShell 5.1 (build 26100) meets the lab requirement of 5.1 or higher.

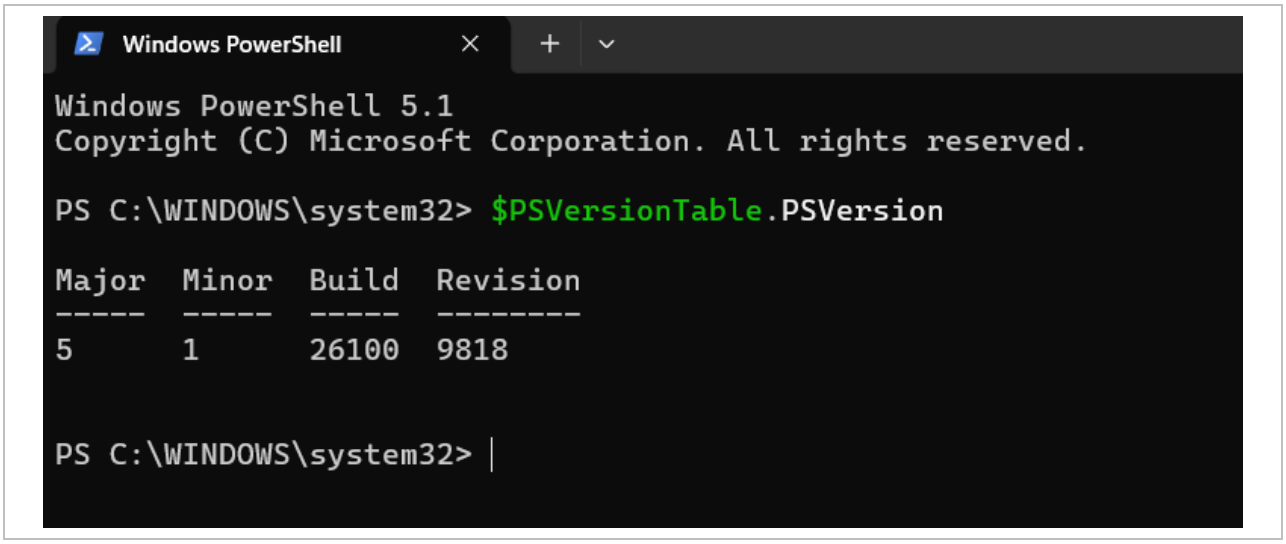


Figure 1: PowerShell version check

2. Basic PowerShell Event Log Queries

Exercise 1: View recent Security events

```
Get-WinEvent -LogName Security -MaxEvents 10
```

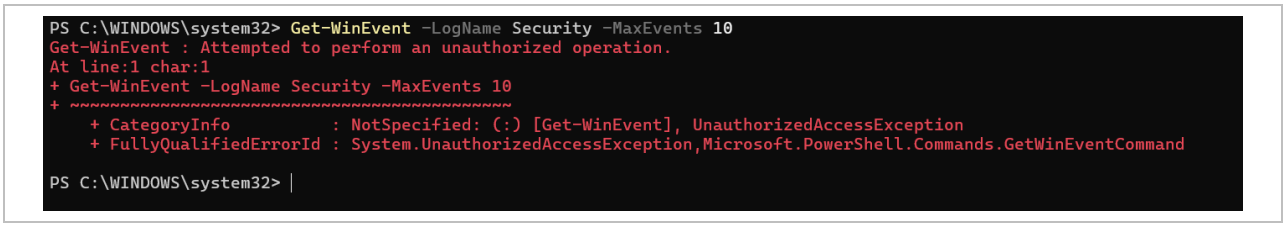


Figure 2: Get-WinEvent returns UnauthorizedAccessException

Observation: Reading the Security log requires Administrator rights (or membership in the *Event Log Readers* group). This error shows the session was **not elevated**, which is the root cause of the empty results in the later exercises.

Exercise 2: Filter by Event ID (4624 successful logons)

```
Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4624} -MaxEvents 20 |
    Select-Object TimeCreated, Id, Message |
    Format-Table -AutoSize
```

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4624} -MaxEvents 20 |
>> Select-Object TimeCreated, Id, Message |
>> Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4624} -MaxEven ...
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 3: Event ID 4624 query: no events found

Observation: A normal Windows machine always has successful logon events, so an empty result indicates the log could not be read, not that no logons happened.

Exercise 3: Filter by time range

Last 24 hours:

```
$StartTime = (Get-Date).AddDays(-1)
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    StartTime=$StartTime
} -MaxEvents 100
```

```
PS C:\WINDOWS\system32> $StartTime = (Get-Date).AddDays(-1)
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     StartTime=$StartTime
>> } -MaxEvents 100
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 4: All Security events, last 24 hours: no events found

Last hour:

```
$StartTime = (Get-Date).AddHours(-1)
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    StartTime=$StartTime
}
```

```
PS C:\WINDOWS\system32> $StartTime = (Get-Date).AddHours(-1)
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     StartTime=$StartTime
>> }
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 5: All Security events, last hour: no events found

Observation: Even queries with **no Event ID filter** returned nothing. This confirms the problem is access to the log, not the specific event IDs.

Exercise 4: Count failed logons (4625), last 24 hours

```
$StartTime = (Get-Date).AddDays(-1)
$FailedLogons = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4625
    StartTime=$StartTime
}
$FailedLogons.Count
```

```
PS C:\WINDOWS\system32> $StartTime = (Get-Date).AddDays(-1)
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> $FailedLogons = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=$StartTime
>> }
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:17
+ $FailedLogons = Get-WinEvent -FilterHashtable @{
+                 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 6: Failed logon count query: no events found

Result: The count could not be determined because no events were returned.

3. Extracting Event Properties

Exercise 5: Understanding event structure

```
$Event = Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4625} -MaxEvents 1
$Event | Format-List *
```

```
PS C:\WINDOWS\system32> $Event = Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>> } -MaxEvents 1
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:10
+ $Event = Get-WinEvent -FilterHashtable @{
+          ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> $Event | Format-List *
```

Figure 7: Retrieving one 4625 event: no events found

Result: `$Event` was empty, so `Format-List *` displayed nothing.

Exercise 6: Extract fields from failed logons

```
Get-WinEvent -FilterHashtable @{LogName='Security'; ID=4625} -MaxEvents 10 |
```

```
    ForEach-Object {
        [PSCustomObject]@{
            TimeCreated   = $_.TimeCreated
            TargetUser    = $_.Properties[5].Value
            SourceIP      = $_.Properties[19].Value
            LogonType     = $_.Properties[8].Value
            FailureReason = $_.Properties[13].Value
        }
    } | Format-Table -AutoSize
```

```
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> $Event | Format-List *Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>> } -MaxEvents 10 |
>> ForEach-Object {
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         TargetUser = $_.Properties[5].Value
>>         SourceIP = $_.Properties[19].Value
>>         LogonType = $_.Properties[8].Value
>>         FailureReason = $_.Properties[13].Value
>>     }
>> } | Format-Table -AutoSize
Format-List : A parameter cannot be found that matches parameter name 'FilterHashtable'.
At line:1 char:36
+ $Event | Format-List *Get-WinEvent -FilterHashtable @{
+                                    ~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument: (:) [Format-List], ParameterBindingException
    + FullyQualifiedErrorId : NamedParameterNotFound,Microsoft.PowerShell.Commands.FormatListCommand
```

Figure 8: Command error caused by two commands joined on one line

Observation: In this attempt two commands were joined on one line (`$Event | Format-List *Get-WinEvent ...`), which caused a `ParameterBindingException` for `FilterHashtable`. **Lesson learned:** each command must be entered on its own line. The correct command is shown above.

Exercise 7: Top 5 source IP addresses

```
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4625
    StartTime=(Get-Date).AddDays(-1)
} | ForEach-Object {
    $_.Properties[19].Value
} | Where-Object { $_ -ne '-' -and $_ -ne '' } |
    Group-Object |
    Sort-Object Count -Descending |
    Select-Object -First 5 Name, Count
```

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $_.Properties[19].Value
>> } | Where-Object {
>>     $_ -ne '-' -and $_ -ne ''
>> } |
>> Group-Object |
>> Sort-Object Count -Descending |
>> Select-Object -First 5 Name, Count
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 9: Top 5 source IPs: no events found

Exercise 8: Top 5 targeted usernames

```
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4625
    StartTime=(Get-Date).AddDays(-1)
} | ForEach-Object {
    $_.Properties[5].Value
} | Where-Object { $_ -ne '-' -and $_ -ne '' } |
    Group-Object |
    Sort-Object Count -Descending |
    Select-Object -First 5 @{Name='Username';Expression={$_.Name}}, @{Name='Failed
Attempts';Expression={$_.Count}} |
    Format-Table -AutoSize
```

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $_.Properties[5].Value
>> } | Where-Object {
>>     $_ -ne '-' -and $_ -ne ''
>> } |
>> Group-Object |
>> Sort-Object Count -Descending |
>> Select-Object -First 5 @{
>>     Name='Username'
>>     Expression={$_.Name}
>> }, @{
>>     Name='Failed Attempts'
>>     Expression={$_.Count}
>> } | Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 10: Top 5 targeted usernames: no events found

Exercise 9: Logon type distribution (failed attempts)

```
Get-WinEvent -FilterHashtable @{
    LogName='Security'
```

```
    ID=4625
    StartTime=(Get-Date).AddDays(-1)
} | ForEach-Object {
    $LogonType = $_.Properties[8].Value
    switch ($LogonType) {
        2  { 'Interactive' }
        3  { 'Network' }
        4  { 'Batch' }
        5  { 'Service' }
        7  { 'Unlock' }
        8  { 'NetworkCleartext' }
        9  { 'NewCredentials' }
        10 { 'RemoteInteractive' }
        11 { 'CachedInteractive' }
        default { "Unknown ($LogonType)" }
    }
} | Group-Object |
    Sort-Object Count -Descending |
    Select-Object @{Name='Logon Type';Expression={$_.Name}},
@{Name='Count';Expression={$_.Count}} |
    Format-Table -AutoSize
```

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     $LogonType = $_.Properties[8].Value
>>     switch ($LogonType) {
>>         2 { 'Interactive' }
>>         3 { 'Network' }
>>         4 { 'Batch' }
>>         5 { 'Service' }
>>         7 { 'Unlock' }
>>         8 { 'NetworkCleartext' }
>>         9 { 'NewCredentials' }
>>         10 { 'RemoteInteractive' }
>>         11 { 'CachedInteractive' }
>>         default { "Unknown ($LogonType)" }
>>     }
>> } | Group-Object |
>> Sort-Object Count -Descending |
>> Select-Object @{
>>     Name='Logon Type'
>>     Expression={$_.Name}
>> }, @{
>>     Name='Count'
>>     Expression={$_.Count}
>> } | Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 11: Logon type distribution: no events found

4. Brute-Force Attack Detection

Detection criteria

- **Threshold:** 5 failed attempts from the same source IP

- **Time window:** 1 hour
- **Indicators:** repeated failures from one IP, multiple usernames targeted, short time window

Exercise 10: Detection script (Detect-BruteForce.ps1)

The script was entered in the console. `-ErrorAction SilentlyContinue` was added so an empty log does not stop the script. The full script is in Appendix A.

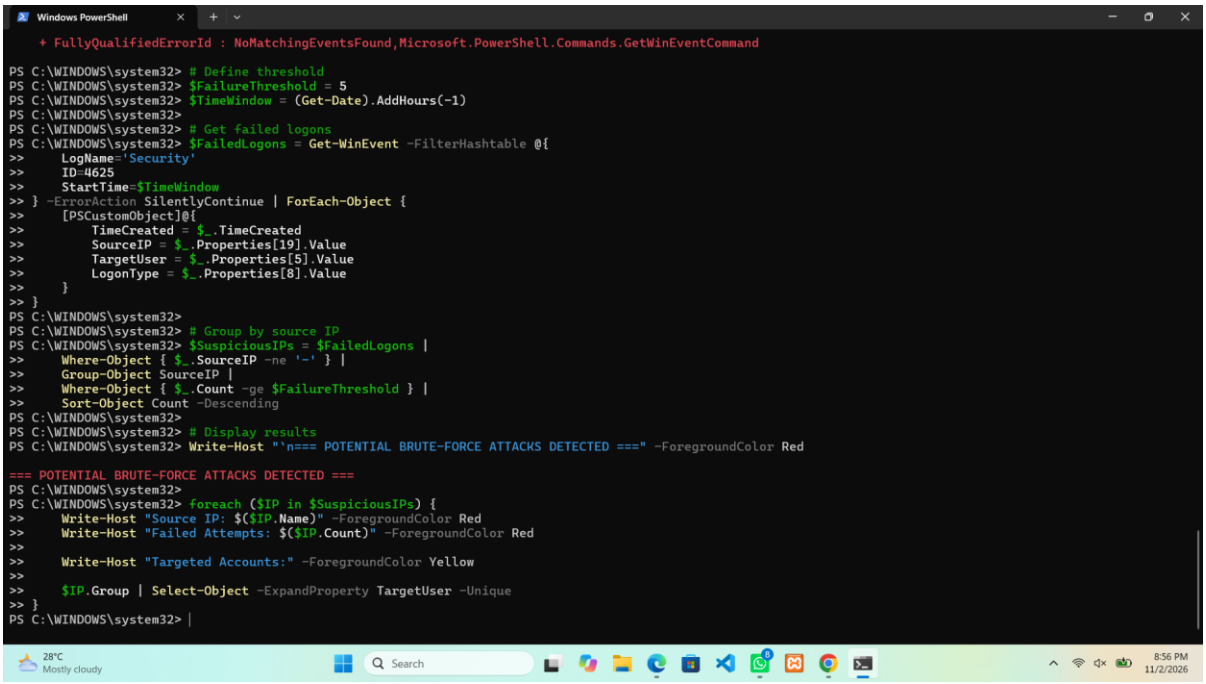


Figure 12: Brute-force detection script and output

Output: Only the heading "POTENTIAL BRUTE-FORCE ATTACKS DETECTED" was printed, with no source IPs listed.

Suspicious IPs identified: None (no data available). This must **not** be read as "no attack": `$FailedLogons` was empty because the Security log could not be read, so `$SuspiciousIPs` was also empty. The heading prints unconditionally; a production version should only print it when `$SuspiciousIPs` contains results.

Exercise 11: Generating test failed logons

To validate the detection logic, failed logons should be generated and verified from an elevated session:

```
1..10 | ForEach-Object {
    net use \\localhost\C$ /user:FakeUser WrongPassword 2>$null
    Start-Sleep -Seconds 2
}

Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4625
    StartTime=(Get-Date).AddMinutes(-5)
```

```
} | Select-Object TimeCreated, Message | Format-List
```

Screenshot to be added after re-running as Administrator.

5. Lateral Movement Indicators

Explicit credential usage (Event ID 4648)

```
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4648
    StartTime=(Get-Date).AddDays(-1)
} | ForEach-Object {
    $xml = [xml]$_.ToXml()
    [PSCustomObject]@{
        TimeCreated  = $_.TimeCreated
        SourceUser   = $xml.Event.EventData.Data[1].'#text'
        TargetUser   = $xml.Event.EventData.Data[5].'#text'
        TargetServer = $xml.Event.EventData.Data[8].'#text'
        Process      = $xml.Event.EventData.Data[11].'#text'
    }
} | Format-Table -AutoSize
```

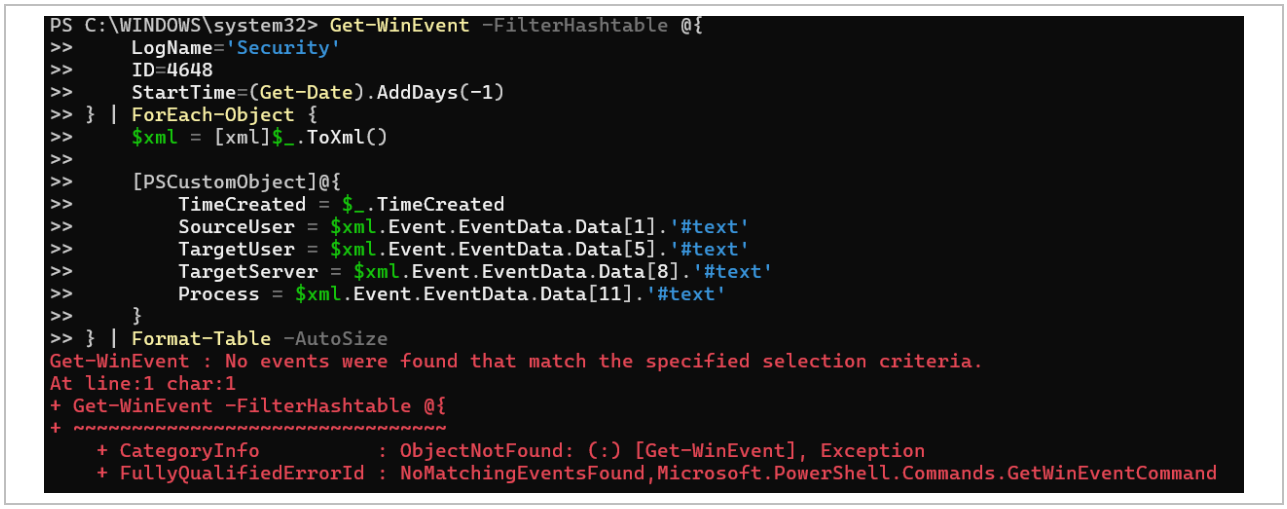


Figure 13: Event ID 4648 query: no events found

Finding: No explicit-credential usage could be confirmed.

Network share access (Event ID 5140)

```
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=5140
    StartTime=(Get-Date).AddHours(-1)
} | ForEach-Object {
    $xml = [xml]$_.ToXml()
    [PSCustomObject]@{
        TimeCreated = $_.TimeCreated
        Account     = $xml.Event.EventData.Data[1].'#text'
```

```
        SourceIP    = $xml.Event.EventData.Data[3].'#text'
        ShareName   = $xml.Event.EventData.Data[4].'#text'
    }
} | Format-Table -AutoSize
```

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=5140
>>     StartTime=(Get-Date).AddHours(-1)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         Account = $xml.Event.EventData.Data[1].'#text'
>>         SourceIP = $xml.Event.EventData.Data[3].'#text'
>>         ShareName = $xml.Event.EventData.Data[4].'#text'
>>     }
>> } | Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 14: Event ID 5140 query: no events found

Finding: No share access events returned. Event 5140 is only generated if *Audit File Share* is enabled.

6. Privilege Escalation Indicators

Admin group changes (Event ID 4732, last 7 days)

```
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4732
    StartTime=(Get-Date).AddDays(-7)
} | ForEach-Object {
    $xml = [xml]$_.ToXml()
    [PSCustomObject]@{
        TimeCreated = $_.TimeCreated
        AddedUser   = $xml.Event.EventData.Data[0].'#text'
        GroupName   = $xml.Event.EventData.Data[2].'#text'
        AddedBy     = $xml.Event.EventData.Data[6].'#text'
    }
} | Format-Table -AutoSize
```

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4732
>>     StartTime=(Get-Date).AddDays(-7)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         AddedUser = $xml.Event.EventData.Data[0].'#text'
>>         GroupName = $xml.Event.EventData.Data[2].'#text'
>>         AddedBy = $xml.Event.EventData.Data[6].'#text'
>>     }
>> } | Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 15: Event ID 4732 query: no events found

Finding: No group membership changes could be confirmed.

Special privilege assignments (Event ID 4672)

```
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4672
    StartTime=(Get-Date).AddHours(-1)
} | ForEach-Object {
    $xml = [xml]$_.ToXml()
    [PSCustomObject]@{
        TimeCreated = $_.TimeCreated
        Account     = $xml.Event.EventData.Data[1].'#text'
        LogonID     = $xml.Event.EventData.Data[3].'#text'
    }
} | Where-Object { $_.Account -notlike '*$' } |
    Format-Table -AutoSize
```

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4672
>>     StartTime=(Get-Date).AddHours(-1)
>> } | ForEach-Object {
>>     $xml = [xml]$_.ToXml()
>>
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         Account = $xml.Event.EventData.Data[1].'#text'
>>         LogonID = $xml.Event.EventData.Data[3].'#text'
>>     }
>> } | Where-Object {
>>     $_.Account -notlike '*$'
>> } | Format-Table -AutoSize
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 16: Event ID 4672 query: no events found

Finding: No results. Event 4672 is normally very frequent on any Windows system (every admin or SYSTEM logon), which again indicates the log was not readable.

7. Troubleshooting and Analysis of Results

Observation	Evidence	Interpretation
Unfiltered `Get-WinEvent -LogName Security` failed with `UnauthorizedAccessException`	Exercise 1	Session lacks permission to read the Security log
All `-FilterHashtable` queries returned `NoMatchingEventsFound`	Exercises 2 to 15	With `-FilterHashtable`, an access-denied condition can surface as "no events found"
Queries with no Event ID filter also returned nothing	Exercise 3	Points to access or log availability, not absence of specific events
Very common events (4624, 4672) missing	Exercises 2, 15	Unusual for an active Windows system

Most likely cause: PowerShell was not running as Administrator. **Other possible causes:** logon auditing disabled in local policy, or the Security log having been cleared (Event ID 1102).

Verification steps to run as Administrator:

```
# Confirm elevation (should return True)
([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Security.Principal.WindowsBuiltInRole]::Administrator)

# Confirm the log is readable
Get-WinEvent -LogName Security -MaxEvents 10

# Check and enable logon auditing
auditpol /get /category:"Logon/Logoff"
auditpol /set /subcategory:"Logon" /success:enable /failure:enable
```

8. Exporting Results

Exercise 17: Export to CSV

```
Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4625
    StartTime=(Get-Date).AddDays(-1)
} | ForEach-Object {
    [PSCustomObject]@{
        TimeCreated   = $_.TimeCreated
        SourceIP      = $_.Properties[19].Value
        TargetUser    = $_.Properties[5].Value
        LogonType     = $_.Properties[8].Value
        FailureReason = $_.Properties[13].Value
```

```
    }
} | Export-Csv -Path "$env:USERPROFILE\Desktop\FailedLogons.csv" -NoTypeInformation
```

```
PS C:\WINDOWS\system32> Get-WinEvent -FilterHashtable @{
>>     LogName='Security'
>>     ID=4625
>>     StartTime=(Get-Date).AddDays(-1)
>> } | ForEach-Object {
>>     [PSCustomObject]@{
>>         TimeCreated = $_.TimeCreated
>>         SourceIP = $_.Properties[19].Value
>>         TargetUser = $_.Properties[5].Value
>>         LogonType = $_.Properties[8].Value
>>         FailureReason = $_.Properties[13].Value
>>     }
>> } | Export-Csv -Path "$env:USERPROFILE\Desktop\FailedLogons.csv" -NoTypeInformation
Get-WinEvent : No events were found that match the specified selection criteria.
At line:1 char:1
+ Get-WinEvent -FilterHashtable @{
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (:) [Get-WinEvent], Exception
    + FullyQualifiedErrorId : NoMatchingEventsFound,Microsoft.PowerShell.Commands.GetWinEventCommand
```

Figure 17: CSV export: no events found, so no CSV was written

Result: The pipeline stopped at `Get-WinEvent`, so no `FailedLogons.csv` was created.

Exercise 18: Export to HTML

```
$HTML = $FailedLogons |
    Select-Object TimeCreated, Id, Message |
    ConvertTo-Html -Title "Failed Logon Report"
$HTML | Out-File "$env:USERPROFILE\Desktop\FailedLogons.html"
```

```
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> $HTML = $FailedLogons |
>> Select-Object TimeCreated, Id, Message |
>> ConvertTo-Html -Title "Failed Logon Report"
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> $HTML | Out-File "$env:USERPROFILE\Desktop\FailedLogons.html"
```

Figure 18: HTML report export commands

Result: No error was displayed, but because `$FailedLogons` was empty the HTML file contains only an empty table with no event rows.

9. Recommendations

Immediate actions

- Re-run all exercises from an **elevated** PowerShell (Win + X, then Terminal (Admin)) and refresh screenshots and exports.
- Verify that *Audit Logon* (success and failure) and *Audit File Share* are enabled so events 4624, 4625 and 5140 are generated.
- Generate test failed logons (Exercise 11) to confirm the detection script flags the source IP.
- Check for Event ID 1102 (audit log cleared) to rule out log tampering.

Long-term improvements

- Add a pre-check to every script that confirms elevation and fails loudly if the Security log cannot be read, so "no access" is never confused with "no attacks".
- Only print the brute-force alert when suspicious IPs actually exist.
- Forward Security events to a SIEM (e.g. Splunk) and alert on repeated 4625 events, 4648 from unusual processes, and 4732 changes to Administrators.
- Enable account lockout policy, enforce MFA, restrict RDP exposure, and increase Security log size and retention.
- Schedule `Analyze-SecurityLogs.ps1` to run daily with Task Scheduler.

Appendix A: Detect-BruteForce.ps1

```
# Define threshold
$FailureThreshold = 5
$TimeWindow = (Get-Date).AddHours(-1)

# Get failed logons
$FailedLogons = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4625
    StartTime=$TimeWindow
} -ErrorAction SilentlyContinue | ForEach-Object {
    [PSCustomObject]@{
        TimeCreated = $_.TimeCreated
        SourceIP    = $_.Properties[19].Value
        TargetUser  = $_.Properties[5].Value
        LogonType   = $_.Properties[8].Value
    }
}

# Group by source IP
$SuspiciousIPs = $FailedLogons |
    Where-Object { $_.SourceIP -ne '-' } |
    Group-Object SourceIP |
    Where-Object { $_.Count -ge $FailureThreshold } |
    Sort-Object Count -Descending

# Display results
Write-Host "`n=== POTENTIAL BRUTE-FORCE ATTACKS DETECTED ===" -ForegroundColor Red
Write-Host "Time Window: Last 1 hour" -ForegroundColor Yellow
Write-Host "Threshold: $FailureThreshold failed attempts`n" -ForegroundColor Yellow

foreach ($IP in $SuspiciousIPs) {
    Write-Host "Source IP: $($IP.Name)" -ForegroundColor Red
    Write-Host "Failed Attempts: $($IP.Count)" -ForegroundColor Red
    Write-Host "Targeted Accounts:" -ForegroundColor Yellow
    $IP.Group | Select-Object -ExpandProperty TargetUser -Unique | ForEach-Object {
        Write-Host "    $_"
    }
    Write-Host ""
}
```

Appendix B: Analyze-SecurityLogs.ps1

```
<#
.SYNOPSIS
    Comprehensive Windows Security Log Analysis Script.
.DESCRIPTION
    Analyzes Windows Security Event Logs for suspicious activity.
.PARAMETER Hours
    Number of hours to analyze (default: 24).
.EXAMPLE
    .\Analyze-SecurityLogs.ps1 -Hours 24
#>
param(
    [int]$Hours = 24
)

$StartTime = (Get-Date).AddHours(-$Hours)

Write-Host "`n========================================" -ForegroundColor Cyan
Write-Host "Windows Security Log Analysis Report" -ForegroundColor Cyan
Write-Host "========================================" -ForegroundColor Cyan
Write-Host "Analysis Period: Last $Hours hours" -ForegroundColor Yellow
Write-Host "Start Time: $StartTime" -ForegroundColor Yellow
Write-Host "========================================`n" -ForegroundColor Cyan

# 1. Failed Logon Analysis
Write-Host "[1] FAILED LOGON ANALYSIS" -ForegroundColor Green
Write-Host "---------------------------------" -ForegroundColor Green

$FailedLogons = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4625
    StartTime=$StartTime
} -ErrorAction SilentlyContinue

if ($FailedLogons) {
    Write-Host "Total Failed Logons: $($FailedLogons.Count)" -ForegroundColor Yellow

    Write-Host "`nTop 5 Source IPs:" -ForegroundColor Cyan
    $FailedLogons | ForEach-Object { $_.Properties[19].Value } |
        Where-Object { $_ -ne '-' } |
        Group-Object | Sort-Object Count -Descending | Select-Object -First 5 |
        Format-Table @{Name='IP Address';Expression={$_.Name}},
@{Name='Attempts';Expression={$_.Count}} -AutoSize

    Write-Host "Top 5 Targeted Accounts:" -ForegroundColor Cyan
    $FailedLogons | ForEach-Object { $_.Properties[5].Value } |
        Where-Object { $_ -ne '-' } |
        Group-Object | Sort-Object Count -Descending | Select-Object -First 5 |
        Format-Table @{Name='Username';Expression={$_.Name}},
@{Name='Attempts';Expression={$_.Count}} -AutoSize
} else {
    Write-Host "No failed logons found." -ForegroundColor Green
}

# 2. Successful Logon Analysis
Write-Host "`n[2] SUCCESSFUL LOGON ANALYSIS" -ForegroundColor Green
Write-Host "---------------------------------" -ForegroundColor Green
```

```
$SuccessfulLogons = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4624
    StartTime=$StartTime
} -ErrorAction SilentlyContinue

if ($SuccessfulLogons) {
    Write-Host "Total Successful Logons: $($SuccessfulLogons.Count)" -ForegroundColor Yellow

    Write-Host "`nLogon Type Distribution:" -ForegroundColor Cyan
    $SuccessfulLogons | ForEach-Object {
        $LogonType = $_.Properties[8].Value
        switch ($LogonType) {
            2  { 'Interactive' }
            3  { 'Network' }
            10 { 'RemoteInteractive' }
            default { "Type $LogonType" }
        }
    } | Group-Object | Sort-Object Count -Descending |
        Format-Table @{Name='Logon Type';Expression={$_.Name}},
@{Name='Count';Expression={$_.Count}} -AutoSize
}

# 3. Account Management
Write-Host "`n[3] ACCOUNT MANAGEMENT EVENTS" -ForegroundColor Green
Write-Host "----------------------------------" -ForegroundColor Green

$AccountEvents = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4720,4722,4724,4732
    StartTime=$StartTime
} -ErrorAction SilentlyContinue

if ($AccountEvents) {
    Write-Host "Account Management Events: $($AccountEvents.Count)" -ForegroundColor Yellow
    $AccountEvents | Group-Object Id |
        Format-Table @{Name='Event ID';Expression={$_.Name}},
@{Name='Count';Expression={$_.Count}} -AutoSize
} else {
    Write-Host "No account management events found." -ForegroundColor Green
}

# 4. Privilege Escalation
Write-Host "`n[4] PRIVILEGE ESCALATION INDICATORS" -ForegroundColor Green
Write-Host "----------------------------------" -ForegroundColor Green

$PrivEsc = Get-WinEvent -FilterHashtable @{
    LogName='Security'
    ID=4672,4648
    StartTime=$StartTime
} -ErrorAction SilentlyContinue

if ($PrivEsc) {
    Write-Host "Privilege-Related Events: $($PrivEsc.Count)" -ForegroundColor Yellow
    $PrivEsc | Group-Object Id |
        Format-Table @{Name='Event ID';Expression={$_.Name}},
@{Name='Count';Expression={$_.Count}} -AutoSize
}
```

```
Write-Host "`n=====================================" -ForegroundColor Cyan
Write-Host "Analysis Complete" -ForegroundColor Cyan
Write-Host "=====================================`n" -ForegroundColor Cyan
```

Lab Completion Time: [Record your time]

Difficulty Level: Intermediate